

```
In [1]: # !pip install langchain
```

```
In [2]: # !pip install openai
```

```
In [3]: import os
        from dotenv import load_dotenv
        load_dotenv()
        OPENAI_API_KEY = os.getenv("OPENAI_API_KEY")
```

```
In [4]: from langchain_classic.chains import LLMChain
        from langchain_classic import PromptTemplate
        from langchain_classic.chat_models import ChatOpenAI

        llm = ChatOpenAI(temperature=0, # 창의성 0으로 설정
                           model_name='gpt-4.1-mini', # 모델명
                           )

        prompt = PromptTemplate(
            input_variables=["country"],
            template= "{country}의 수도는 어디야?",
        )

        chain = LLMChain(llm=llm, prompt=prompt) #프롬프트와 모델을 체인으로 연결
        chain.run("대한민국")
```

C:\Users\User\AppData\Local\Temp\ipykernel_11328\2716027567.py:5: LangChainDeprecationWarning: The class `ChatOpenAI` was deprecated in LangChain 0.0.10 and will be removed in 1.0. An updated version of the class exists in the `langchain-openai` package and should be used instead. To use it run `pip install -U `langchain-openai` and import as `from `langchain_openai import ChatOpenAI``.

```
llm = ChatOpenAI(temperature=0, # 창의성 0으로 설정
```

C:\Users\User\AppData\Local\Temp\ipykernel_11328\2716027567.py:15: LangChainDeprecationWarning: The class `LLMChain` was deprecated in LangChain 0.1.17 and will be removed in 1.0. Use `RunnableSequence`, e.g., `prompt | llm`` instead.

```
chain = LLMChain(llm=llm, prompt=prompt) #프롬프트와 모델을 체인으로 연결
```

C:\Users\User\AppData\Local\Temp\ipykernel_11328\2716027567.py:16: LangChainDeprecationWarning: The method `Chain.run` was deprecated in langchain-classic 0.1.0 and will be removed in 1.0. Use `invoke` instead.

```
chain.run("대한민국")
```

Out[4]: '대한민국의 수도는 서울입니다.'

```
In [5]: #프롬프트1 정의
prompt1 = PromptTemplate(
    input_variables=['sentence'],
    template="다음 문장을 한글로 번역하세요.\n\n{sentence}"
)
#번역(체인1)에 대한 모델
chain1 = LLMChain(llm=llm, prompt=prompt1, output_key="translation")

#프롬프트2 정의
prompt2 = PromptTemplate.from_template(
    "다음 문장을 한 문장으로 요약하세요.\n\n{translation}"
)
#요약(체인2)에 대한 모델
chain2 = LLMChain(llm=llm, prompt=prompt2, output_key="summary")

from langchain_classic.chains import SequentialChain
all_chain = SequentialChain(
    chains=[chain1, chain2],
    input_variables=['sentence'],
    output_variables=['translation', 'summary'],
)
sentence="""
One limitation of LLMs is their lack of contextual information
(e.g., access to some specific documents or emails).
You can combat this by giving LLMs access to the specific external data.
For this, you first need to load the external data with a document loader.
LangChain provides a variety of loaders for different types of documents ranging
from PDFs and emails to websites and YouTube videos.
"""
all_chain(sentence)
```

C:\Users\User\AppData\Local\Temp\ipykernel_11328\2399858834.py:30: LangChainDeprecationWarning: The method `Chain.\_\_call\_\_` was deprecated in langchain-classic 0.1.0 and will be removed in 1.0. Use `invoke` instead.

```
all_chain(sentence)
```

```
Out[5]: {'sentence': '\n One limitation of LLMs is their lack of contextual information\n (e.g., access to some specific documents or emails). \n You can combat this by giving LLMs access to the specific external data.\n For this, you first need to load the external data with a document loader.\n LangChain provides a variety of loaders for different types of documents ranging\n from PDFs and emails to websites and YouTube videos.\n',  
        'translation': 'LLM의 한 가지 한계는 특정 문서나 이메일과 같은 맥락 정보를 갖고 있지 않다는 점입니다. \n이를 해결하려면 LLM이 특정 외부 데이터에 접근할 수 있도록 해야 합니다. \n이를 위해 먼저 문서 로더를 사용해 외부 데이터를 불러와야 합니다. \nLangChain은 PDF, 이메일부터 웹사이트, 유튜브 동영상에 이르기까지 다양한 유형의 문서를 위한 여러 로더를 제공합니다.',  
        'summary': 'LLM의 한계를 극복하기 위해 LangChain이 제공하는 다양한 문서 로더를 사용해 PDF, 이메일, 웹사이트, 유튜브 동영상 등 외부 데이터를 불러와 LLM이 해당 맥락 정보를 활용할 수 있도록 해야 합니다.'}
```

```
In [ ]:
```